

Vau

Manual técnico

Servidor de CI/CD em PHP que recebe webhooks do GitHub e do GitLab e roda testes e deploy em containers Docker isolados. Instalação, configuração, pipeline, cada tela, arquitetura, segurança, API e operação, passo a passo.

Versão 1.0.0
Outubro de 2026

Copyright (©) 2026 Erik Pimenta

Sumário

01 Visão geral 4

1.1 Para que serve 4
1.2 O que o projeto entrega 5
1.3 Funcionalidades 5
1.4 Identidade visual 6

02 Instalação 7

2.1 Pré-requisitos 7
2.2 Caminho rápido 7
2.3 Passo a passo manual 8
2.4 Docker Compose com PostgreSQL 8
2.5 Primeiro acesso 9

03 Configuração 10

3.1 Todas as variáveis 10
3.2 A chave dos segredos 11
3.3 Runtime local 11

04 Conectando repositórios 12

4.1 Cadastrando o projeto 12
4.2 GitHub 12
4.3 GitLab 13
4.4 O que gera build 13
4.5 Filtros e cancelamento automático 13
4.6 Status no commit, avisos e selo 14

05 O arquivo .vau.yml 15

5.1 Exemplo completo 15
5.2 Todas as chaves 15
5.3 Variáveis disponíveis nas etapas 16
5.4 Exemplos por linguagem 17

06 Usando o painel 18

6.1 Painel 18
6.2 Projetos 19
6.3 Build e log ao vivo 21
6.4 Entregas de webhook 22
6.5 Sistema e conta 23
6.6 Tema escuro e celular 24

Copyright (©) 2026 Erik Pimenta

07 Arquitetura 26
7.1 Módulos do código 26
7.2 Como um build é isolado 27
7.3 Fila, workers e cancelamento 27
7.4 Executando um build, passo a passo 28

08 Segurança 29
8.1 Proteções 29
8.2 Boas práticas de implantação 29

09 API REST 30
9.1 Rotas 30
9.2 Exemplos com curl 31
9.3 Códigos de resposta 31

10 Operação 33
10.1 Linha de comando 33
10.2 Saúde e métricas 34
10.3 Logs 34
10.4 Limpeza e backup 34

11 Qualidade e manutenção 35
11.1 Testes automatizados 35
11.2 Integração contínua 35
11.3 Commit automático diário 36
11.4 Solução de problemas 36

Copyright (©) 2026 Erik Pimenta

01

Visão geral

O Vau é um servidor de integração e entrega contínuas (CI/CD) escrito em PHP. Ele recebe os webhooks do GitHub e do GitLab, clona o commit exato e roda os testes e o deploy em containers Docker isolados, mostrando tudo num painel com log ao vivo.

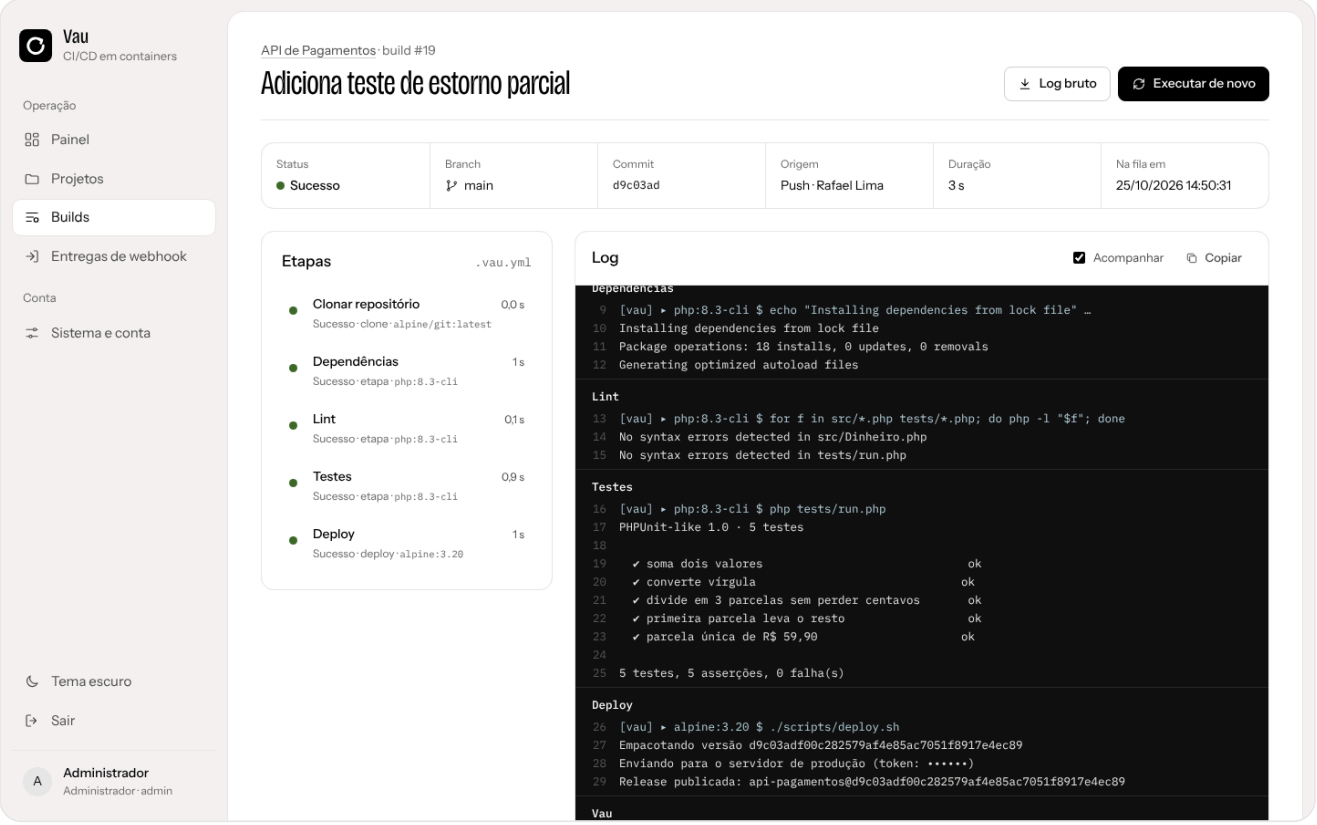


Figura 1.1 Build terminado: etapas à esquerda, log de cada etapa à direita, com o segredo de deploy mascarado.

1.1 Para que serve

- Rodar os testes de cada push e de cada pull/merge request antes do merge, com o resultado aparecendo ao lado do commit no GitHub ou no GitLab.
- Publicar automaticamente o que chega à branch principal (ou a uma tag de versão) só quando todas as etapas passaram.
- Ter um CI próprio, no seu servidor, sem limite de minutos e sem enviar código ou segredos para serviços de terceiros.
- Dar à equipe um histórico auditável: quem disparou, qual commit, quanto tempo levou, o log completo e por que falhou.

Copyright (©) 2026 Erik Pimenta

1.2 O que o projeto entrega

O Vau vem pronto para instalar: painel, API, worker, banco de dados, imagem Docker, scripts de inicialização, testes e documentação. Nenhum repositório ou dado real vem junto. A aplicação não depende de nenhum pacote externo do Composer para funcionar; o Composer só é usado para as ferramentas de qualidade (PHPUnit, PHPStan e PHP-CS-Fixer).

Camada	Tecnologia	Papel
Painel	PHP 8.2+, templates PHP, CSS e JavaScript sem dependências	Painel, projetos, builds, log ao vivo, entregas de webhook, sistema e conta
API	Rotas REST em JSON com token Bearer	Disparar, cancelar e repetir builds, consultar projetos, builds, logs e números
Webhooks	GitHub (HMAC SHA-256) e GitLab (X-Gitlab-Token)	Push, tag, pull request e merge request
Worker	Processo PHP de longa duração, um ou vários	Consome a fila e executa os builds
Execução	Docker Engine (CLI)	Um volume por build, um container novo por etapa, limites e permissões mínimas
Banco	SQLite (padrão), MySQL, MariaDB ou PostgreSQL via PDO	Projetos, fila, builds, logs, sessões, auditoria
Criptografia	libsodium (secretbox), Argon2id	Segredos cifrados em repouso e senhas com hash
Qualidade	PHPUnit, PHPStan, PHP-CS-Fixer, GitHub Actions	Testes, análise estática, estilo e CI

1.3 Funcionalidades

- **Webhooks verificados** do GitHub e do GitLab, com registro de toda entrega (aceita ou recusada), proteção contra entregas repetidas e reenvio pelo painel.
- **Pipeline em YAML** (`.vau.yml`) versionado no próprio repositório, ou um pipeline padrão cadastrado no projeto.
- **Isolamento**: cada etapa roda num container descartável com CPU, memória e processos limitados, sem novos privilégios e com rede opcional.
- **Deploy condicionado** à branch ou à tag (`main`, `tag:v*`), só depois de todas as etapas passarem.
- **Segredos** cifrados no banco, entregues como variáveis de ambiente, mascarados no log e nunca enviados a pull requests de forks. Segredos "somente deploy" não chegam às etapas de teste.
- **Log ao vivo** separado por etapa, com acompanhamento automático, cópia e download do log bruto.
- **Fila no banco** com vários workers, cancelamento de builds antigos da mesma branch, cancelamento manual e repetição.
- **Status no commit** pela API do GitHub ou do GitLab, selo SVG para o README e avisos em Slack, Discord ou Google Chat.
- **Contas e perfis** (administrador, operador, leitor), tokens de API, bloqueio por tentativas e trilha de auditoria.

Copyright (©) 2026 Erik Pimenta

- **Operação**: `/health`, `/metrics` no formato Prometheus, logs em JSON Lines, limpeza de histórico e diagnóstico pela linha de comando.
- **Tema claro e escuro** e layout para celular.

1.4 Identidade visual

O painel segue a identidade "caixa única" dos outros projetos da família: fundo, menu e cabeçalhos impressos na mesa em `#F5F4F4`, o conteúdo dentro de uma só caixa branca dividida por paredes finas, os números principais na tampa preta, títulos e números em Bricolage Grotesque fina e condensada, textos em Instrument Sans, código e logs em IBM Plex Mono, e o status sempre como um ponto de cor. Paleta: `#FFFFFF`, `#4D4B4B`, `#000000`, `#333232` e `#BAB7B6`. A logo, em preto e branco, é uma volta que se fecha numa seta: o commit que atravessa o vau e chega do outro lado.

Copyright (©) 2026 Erik Pimenta

02
Instalação

Escolha entre o caminho rápido (um script), a instalação manual ou o Docker Compose com PostgreSQL. Em todos, o primeiro acesso usa o mesmo usuário administrador.

2.1 Pré-requisitos

Item	Versão	Para quê
PHP	8.2 ou mais recente	Painel, API e worker. Extensões: `pdo` com o driver do banco (`pdo_sqlite` por padrão), `sodium` , `mbstring` , `curl` , `json`
Extensão pcntl	opcional	Desligamento gracioso do worker e batimento durante builds longos (Linux e macOS)
Docker Engine	20.10 ou mais recente	Na máquina do worker. O painel não precisa de Docker
Banco	SQLite 3.35+, MySQL 8+, MariaDB 10.6+ ou PostgreSQL 13+	SQLite já vem com o PHP e atende um servidor
Composer	2.x, opcional	Só para rodar testes, análise estática e estilo

> O worker precisa conversar com o Docker do servidor. Quem tem acesso ao Docker tem, na prática, acesso de administrador à máquina. Rode o worker num servidor dedicado aos builds, nunca no mesmo servidor de produção das suas aplicações.

2.2 Caminho rápido

1. Baixe o projeto (`git clone` ou o ZIP do GitHub).
2. **Linux e macOS:** rode `./iniciar.sh` . **Windows:** dê dois cliques em `iniciar.cmd` .
3. O script confere o PHP, cria o `.env` , a chave, o banco e o usuário `admin` , inicia um worker se o Docker responder e sobe o painel em **http://127.0.0.1:8080**.

No Windows o worker abre numa segunda janela e precisa do Docker Desktop aberto. Sem Docker, o painel funciona normalmente e os builds ficam na fila até um worker aparecer.

Copyright (©) 2026 Erik Pimenta

2.3 Passo a passo manual

```
git clone https://github.com/ravelroot/Containerized-CI-CD-System.git vau
cd vau
cp .env.example .env            # Windows: copy .env.example .env
php bin/vau setup               # pastas, chave, banco e usuário admin
php bin/vau doctor              # confere PHP, extensões, banco, Docker
php bin/vau serve               # painel em http://127.0.0.1:8080
php bin/vau worker              # em outro terminal: consome a fila
```

Em produção, aponte um servidor web para a pasta `public/` (Apache com `FallbackResource /index.php` ou Nginx com `try_files $uri /index.php?$query_string`) e mantenha o worker como serviço (systemd, Supervisor ou o próprio Docker). O `serve` usa o servidor embutido do PHP com quatro processos e serve para testes e uso pequeno.

```
# /etc/systemd/system/vau-worker@.service  (vários workers: vau-worker@1, @2...)
[Unit]
Description=Vau worker %i
After=docker.service

[Service]
WorkingDirectory=/opt/vau
ExecStart=/usr/bin/php bin/vau worker
Restart=always
KillSignal=SIGTERM
TimeoutStopSec=120
User=vau            # usuário no grupo docker

[Install]
WantedBy=multi-user.target
```

2.4 Docker Compose com PostgreSQL

O `docker-compose.yml` sobe três serviços: `web` (Apache + PHP na porta 8080), `worker` (a mesma imagem, com o socket do Docker do host) e `db` (PostgreSQL 16). Os dois primeiros dividem o volume `vau-storage`, onde fica a chave.

```
POSTGRES_PASSWORD=troque-esta-senha docker compose up -d --build
docker compose logs -f worker
docker compose up -d --scale worker=3     # mais builds em paralelo
```

O `entrypoint` da imagem roda `php bin/vau setup` antes de subir, então banco, chave e usuário admin são criados sozinhos. O worker tem `stop_grace_period` de 2 minutos para terminar o build em andamento quando o container é parado.

Copyright (©) 2026 Erik Pimenta

2.5 Primeiro acesso

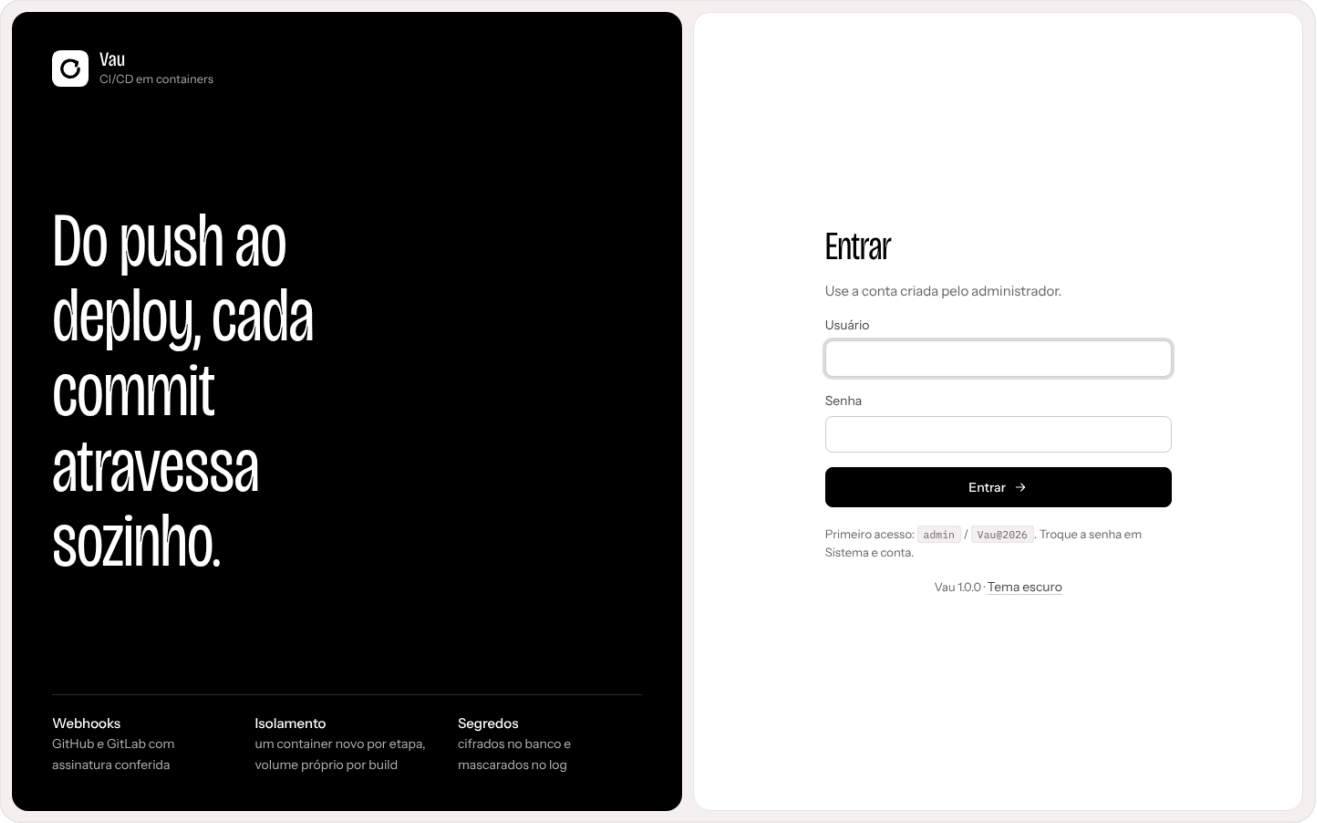


Figura 2.1 Tela de entrada. Do lado preto, o que o Vau garante; do lado branco, o formulário.

Usuário	Senha	Perfil
`admin`	`Vau@2026`	Administrador

Troque a senha em **Sistema e conta** logo no primeiro acesso. Para outra senha inicial, rode o setup com `VAU_ADMIN_PASSWORD=...`. Esqueceu a senha? `php bin/vau user:password admin` no servidor redefine a senha e remove bloqueios.

Copyright (©) 2026 Erik Pimenta

03

Configuração

Toda a configuração fica no arquivo .env (modelo em .env.example). Variáveis do ambiente, como as do Docker ou do systemd, têm prioridade sobre o arquivo.

3.1 Todas as variáveis

Variável	Padrão	Função
`VAU_APP_URL`	`http://127.0.0.1:8080`	Endereço público do painel. Aparece na URL do webhook, nos links do status no commit e nos avisos
`VAU_TIMEZONE`	`America/Sao_Paulo`	Fuso das datas exibidas. O banco guarda tudo em UTC
`VAU_DATABASE_URL`	`sqlite:storage/vau.sqlite`	Banco: `sqlite:` , `mysql://` , `mariadb://` ou `pgsql://usuario:senha@host:porta/banco`
`VAU_APP_KEY`	vazia	Chave (base64 de 32 bytes) que cifra os segredos. Vazia: criada em `storage/.chave` . Gere com `php bin/vau key:generate`
`VAU_AUTO_MIGRATE`	`true`	Aplica migrações pendentes na primeira conexão. Desligue para migrar só no deploy
`VAU_RUNTIME`	`docker`	`docker` (isolado) ou `local` (sem isolamento, só para desenvolvimento)
`VAU_DOCKER_BIN`	`docker`	Caminho do cliente Docker. Use `DOCKER_HOST` para um daemon remoto
`VAU_CLONE_IMAGE`	`alpine/git:latest`	Imagem do clone e da leitura do pipeline. Fixe uma versão em produção
`VAU_DEFAULT_IMAGE`	`alpine:3.20`	Imagem das etapas quando o pipeline não informa `image`
`VAU_BUILD_TIMEOUT`	`30`	Tempo limite, em minutos, sugerido para projetos novos
`VAU_CPUS`	`1.0`	CPUs sugeridas para projetos novos
`VAU_MEMORY`	`1g`	Memória sugerida para projetos novos
`VAU_NETWORK`	`bridge`	Rede sugerida para projetos novos
`VAU_PIDS_LIMIT`	`512`	Máximo de processos por container de etapa (`--pids-limit`)
`VAU_DROP_CAPABILITIES`	`true`	Remove todas as capacidades do kernel e devolve só CHOWN, DAC_OVERRIDE, FOWNER, SETGID e SETUID

Copyright (©) 2026 Erik Pimenta

Variável	Padrão	Função
`VAU_CLONE_DEPTH`	`50`	Profundidade do `git fetch`
`VAU_WORKSPACE_DIR`	`storage/workspaces`	Pasta dos builds no runtime local
`VAU_MAX_LOG_MB`	`10`	Tamanho máximo do log de um build; o excedente é cortado com aviso
`VAU_LOG_RETENTION_DAYS`	`30`	Idade usada por `php bin/vau prune`
`VAU_LOG_LEVEL`	`info`	`debug`, `info`, `warning` ou `error`
`VAU_LOG_FILE`	`storage/logs/vau.log`	Arquivo do log técnico em JSON Lines
`VAU_MAX_PAYLOAD_KB`	`2048`	Tamanho máximo do corpo de um webhook (413 acima disso)
`VAU_WEBHOOK_RATE_LIMIT`	`120`	Webhooks por minuto por IP (429 acima disso)
`VAU_LOGIN_MAX_ATTEMPTS`	`5`	Senhas erradas seguidas até o bloqueio da conta
`VAU_LOGIN_LOCK_MINUTES`	`15`	Duração do bloqueio
`VAU_SESSION_MINUTES`	`480`	Tempo de inatividade até a sessão expirar
`VAU_METRICS_TOKEN`	vazia	Se definido, `/metrics` exige `Authorization: Bearer` com este valor
`VAU_WORKER_SLEEP`	`2`	Segundos entre consultas à fila quando ela está vazia
`VAU_WORKER_STALE_SECONDS`	`120`	Sem batimento por esse tempo, o build do worker é encerrado com erro
`VAU_ADMIN_PASSWORD`	`Vau@2026`	Senha do admin criado pelo `setup` (só na primeira vez)
`VAU_ALLOW_FILE_REPOS`	vazia	Com `1`, aceita URLs `file://`. Só para desenvolvimento e testes

A tela **Sistema e conta** mostra a configuração ativa (sem a chave e com a senha do banco mascarada).

3.2 A chave dos segredos

Todos os segredos (segredo do webhook, token de acesso ao provedor e variáveis secretas) são cifrados com a chave antes de ir para o banco. Com um único servidor, o arquivo `storage/.chave` basta. Com vários servidores do painel ou workers em máquinas diferentes, gere uma chave e use a mesma `VAU_APP_KEY` em todos. Guarde a chave num cofre: sem ela, os segredos gravados não podem ser lidos.

3.3 Runtime local

Com `VAU_RUNTIME=local`, o worker roda as etapas direto na máquina, numa pasta em `storage/workspaces`, com `sh -ec`. É útil para desenvolver o próprio Vau ou para demonstração sem Docker. Não há isolamento nenhum: um script do repositório pode ler e apagar arquivos do servidor. Use só com repositórios em que você confia.

Copyright (©) 2026 Erik Pimenta

04

Conectando repositórios

Cada projeto do Vau corresponde a um repositório. O cadastro gera uma URL e um segredo exclusivos para o webhook.

4.1 Cadastrando o projeto

1. Em **Projetos** → **Novo projeto**, informe o nome, o provedor e a URL HTTPS de clonagem (`https://github.com/org/repo.git`).
2. Defina a branch padrão (usada nos builds manuais e no selo).
3. Para repositório privado, informe um token de acesso: no GitHub, um token fine-grained com leitura de conteúdo e escrita de commit statuses; no GitLab, um token com `read_repository` e `api`.
4. Escolha os gatilhos, o filtro de branches, os limites do container, o deploy e os avisos.
5. Ao salvar, o Vau abre a aba **Webhook** com a URL e o segredo.

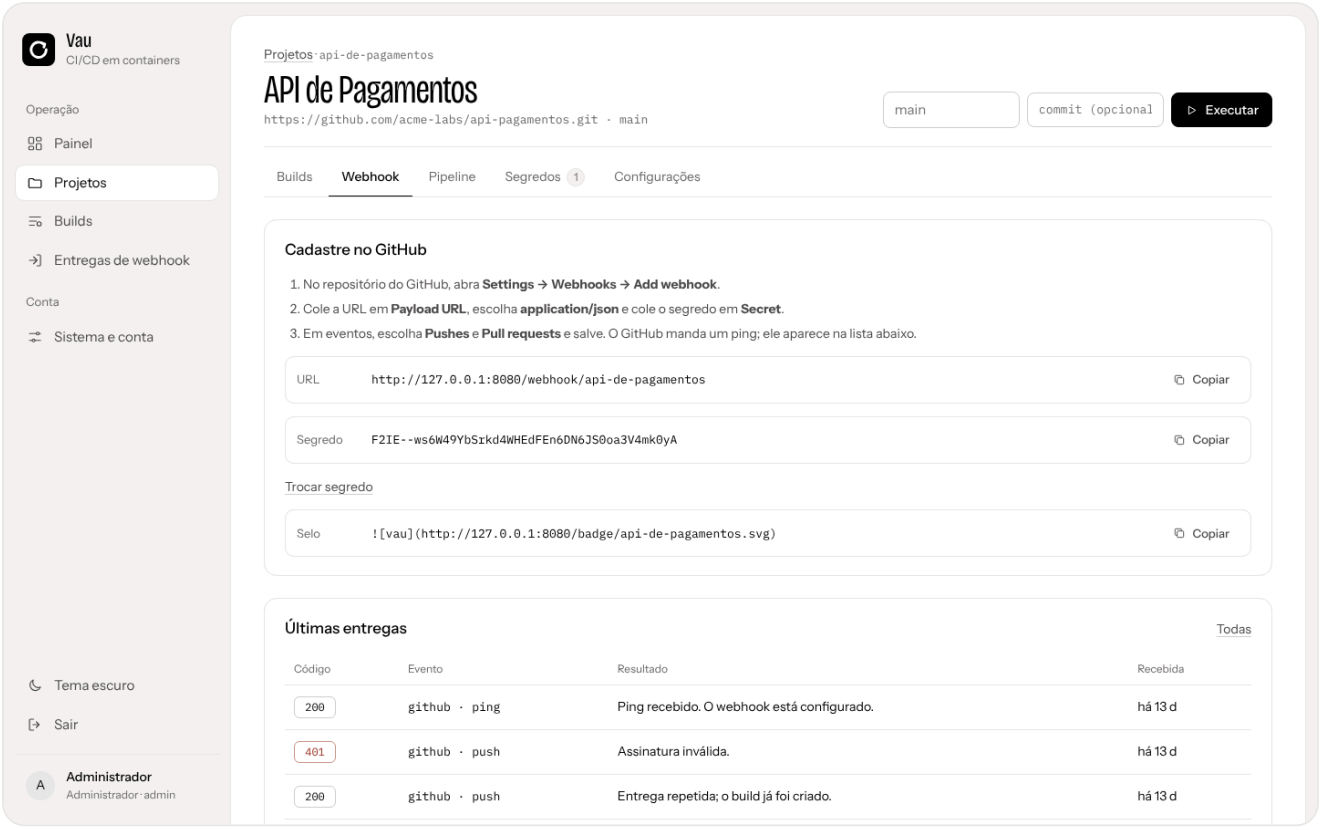


Figura 4.1 Aba Webhook: instruções do provedor, URL, segredo (borrado até o clique), selo e as últimas entregas.

4.2 GitHub

1. No repositório, abra **Settings** → **Webhooks** → **Add webhook**.

Copyright (©) 2026 Erik Pimenta

2. Em **Payload URL**, cole a URL do Vau (`https://ci.suaempresa.com/webhook/{slug}`). Em **Content type**, escolha `application/json`.
3. Em **Secret**, cole o segredo do projeto.
4. Em eventos, escolha **Pushes** e **Pull requests**. Salve.
5. O GitHub envia um `ping`, que aparece em **Entregas de webhook** com a resposta "Ping recebido".

O Vau confere o cabeçalho `X-Hub-Signature-256` (HMAC SHA-256 do corpo bruto com o segredo) em tempo constante, e usa `X-GitHub-Delivery` para ignorar entregas repetidas.

4.3 GitLab

1. No projeto, abra **Settings** → **Webhooks** → **Add new webhook**.
2. Cole a URL do Vau e, em **Secret token**, o segredo.
3. Marque **Push events**, **Tag push events** e **Merge request events**. Salve e use **Test** para conferir.

O GitLab envia o segredo no cabeçalho `X-Gitlab-Token`, comparado em tempo constante. Entregas repetidas são reconhecidas por `X-Gitlab-Event-UUID`.

4.4 O que gera build

Evento recebido	Gera build quando	Referência clonada
GitHub `push` em branch	commit novo (branch apagada é ignorada)	`refs/heads/<branch>`
GitHub `push` de tag	tag criada	`refs/tags/<tag>`
GitHub `pull_request`	`opened`, `synchronize`, `reopened`, `ready_for_review`	`refs/pull/<n>/head`
GitLab `Push Hook` / `Tag Push Hook`	commit ou tag novos	`refs/heads/...` ou `refs/tags/...`
GitLab `Merge Request Hook`	`open`, `reopen` ou `update` com commits novos	`refs/merge-requests/<n>/head`
`ping` e outros eventos	nunca (respondem 200 e ficam registrados)	—

O clone sempre usa a URL cadastrada no projeto, nunca a que vem no corpo do webhook, e faz checkout do commit exato informado pelo provedor.

4.5 Filtros e cancelamento automático

- **Gatilhos**: liga ou desliga push, pull/merge request e tag por projeto.
- **Filtro de branches**: padrões separados por vírgula com curingas (`main, develop, release/*`). Um padrão com `!` exclui (`!wip/*`). Vazio aceita todas. Para tags, o filtro compara o nome da tag.
- **Cancelar builds antigos**: quando chega um commit novo numa branch, os builds dela que ainda estão na fila são cancelados e os que estão rodando recebem pedido de cancelamento.

Copyright (©) 2026 Erik Pimenta

4.6 Status no commit, avisos e selo

Com o token de acesso e a opção **Publicar o status no commit**, o Vau marca o commit como pendente, sucesso ou falha, com o contexto `vau/<slug>` e link para o build (API de commit statuses do GitHub, inclusive GitHub Enterprise em `/api/v3`, ou do GitLab). O campo **Webhook de chat** recebe um JSON com `text` e `content`, aceito por Slack, Google Chat e Discord; escolha avisar em falhas e recuperações, em todos os builds ou nunca. O selo fica em `/badge/<slug>.svg` (aceita `?branch=`) e pode ir no README do repositório.

Copyright (©) 2026 Erik Pimenta

05

O arquivo .vau.yml

O pipeline fica na raiz do repositório e acompanha o código: cada branch pode ter o seu. O Vau procura .vau.yml, .vau.yaml e .vau.json, nesta ordem; se não achar, usa o pipeline padrão do projeto.

5.1 Exemplo completo

```
image: php:8.3-cli          # imagem padrão das etapas
timeout: 20                 # minutos; vale o menor entre este e o do projeto
env:
  APP_ENV: testing
steps:
  - name: Dependências
    run: composer install --no-interaction --prefer-dist
  - name: Testes
    run: vendor/bin/phpunit
  - name: Front
    image: node:22-alpine   # imagem própria da etapa
    run:
      - npm ci
      - npm run build
  - name: Auditoria
    run: composer audit
    allow_failure: true     # falha aqui não derruba o build
  - name: Testes lentos
    run: vendor/bin/phpunit --group lento
    branches: [main, "release/*"]
deploy:
  branches: [main, "tag:v*"]
  image: alpine:3.20
  run: ./scripts/deploy.sh
```

5.2 Todas as chaves

Chave	Tipo	Regra
`image`	texto	Imagem Docker padrão (`nome:tag` ou com `@sha256:`). Sem ela, vale `VAU_DEFAULT_IMAGE`
`timeout`	inteiro	Minutos, de 1 a 360. O build usa o menor valor entre este e o do projeto
`env`	mapa	Variáveis para todas as etapas. Nomes com letras, números e sublinhado
`steps`	lista	Obrigatória, de 1 a 30 etapas, executadas em ordem. Cada item pode ser só um comando (texto) ou um mapa

Copyright (©) 2026 Erik Pimenta

Chave	Tipo	Regra
`steps[].name`	texto	Nome exibido (até 80 caracteres). Sem nome: "Etapa N"
`steps[].run`	texto ou lista	Script executado com `/bin/sh -e` na pasta `/workspace`. Uma lista vira linhas do mesmo script. Até 20 mil caracteres
`steps[].image`	texto	Imagem só desta etapa
`steps[].env`	mapa	Variáveis só desta etapa (somadas às gerais)
`steps[].branches`	lista	A etapa só roda nessas branches; nas outras aparece como pulada
`steps[].allow_failure`	booleano	Se `true`, a falha é registrada e o build continua
`deploy`	mapa	Etapa final, só quando todas as anteriores passaram. Aceita `name`, `run`, `image`, `env` e `branches`
`deploy.branches`	lista	Branches liberadas (padrão: `[main]`). Tags entram com o prefixo `tag:`, como `"tag:v*"`

Chaves desconhecidas, imagens com caracteres inválidos e nomes de variável inválidos fazem o build terminar com **erro** e a explicação na tela, antes de qualquer etapa rodar. O deploy nunca roda em pull/merge request nem quando o projeto desliga a opção **Permitir a etapa de deploy**.

O leitor de YAML cobre o que pipelines usam: mapas e listas por indentação, listas de mapas, textos com e sem aspas, blocos `|` e `>`, listas e mapas em linha e comentários. Âncoras, tags e vários documentos no mesmo arquivo não são aceitos, e a indentação deve usar espaços.

5.3 Variáveis disponíveis nas etapas

Variável	Conteúdo
`CI`, `VAU`	`true`
`VAU_VERSION`	Versão do Vau
`VAU_BUILD_ID`, `VAU_BUILD_NUMBER`	Identificador global e número do build no projeto
`VAU_BUILD_URL`	Link do build no painel
`VAU_PROJECT`	Slug do projeto
`VAU_TRIGGER`	`push`, `pull_request`, `tag`, `manual`, `retry` ou `api`
`VAU_REF`, `VAU_BRANCH`, `VAU_TAG`	Referência clonada, branch e tag
`VAU_COMMIT`	SHA do commit
`VAU_PR`	Número do pull/merge request, quando houver
`VAU_DEPLOY`	`true`, só na etapa de deploy
Segredos do projeto	Com o nome cadastrado (ex.: `DEPLOY_TOKEN`). Os marcados "somente deploy" só existem no deploy

Copyright (©) 2026 Erik Pimenta

5.4 Exemplos por linguagem

Node.js

```
image: node:22-alpine
steps:
  - name: Instalar
    run: npm ci
  - name: Lint e testes
    run:
      - npm run lint
      - npm test -- --ci
deploy:
  branches: [main]
  run: npx wrangler deploy     # usa o segredo CLOUDFLARE_API_TOKEN
```

Python

```
image: python:3.12-slim
env:
  PIP_DISABLE_PIP_VERSION_CHECK: "1"
steps:
  - name: Dependências
    run: pip install -r requirements.txt
  - name: Testes
    run: python -m pytest -q
```

Imagem Docker publicada a cada tag

```
image: alpine:3.20
steps:
  - name: Testes
    image: golang:1.23
    run: go test ./...
deploy:
  branches: ["tag:v*"]
  image: alpine:3.20
  run: |
    apk add --no-cache curl
    curl -fsS -X POST -H "Authorization: Bearer $DEPLOY_TOKEN" \
      "https://deploy.suaempresa.com/api/release?versao=$VAU_TAG"
```

Copyright (©) 2026 Erik Pimenta

06

Usando o painel

O painel tem cinco áreas no menu lateral: Painel, Projetos, Builds, Entregas de webhook e Sistema e conta. O que cada pessoa vê e pode fazer depende do perfil.

6.1 Painel

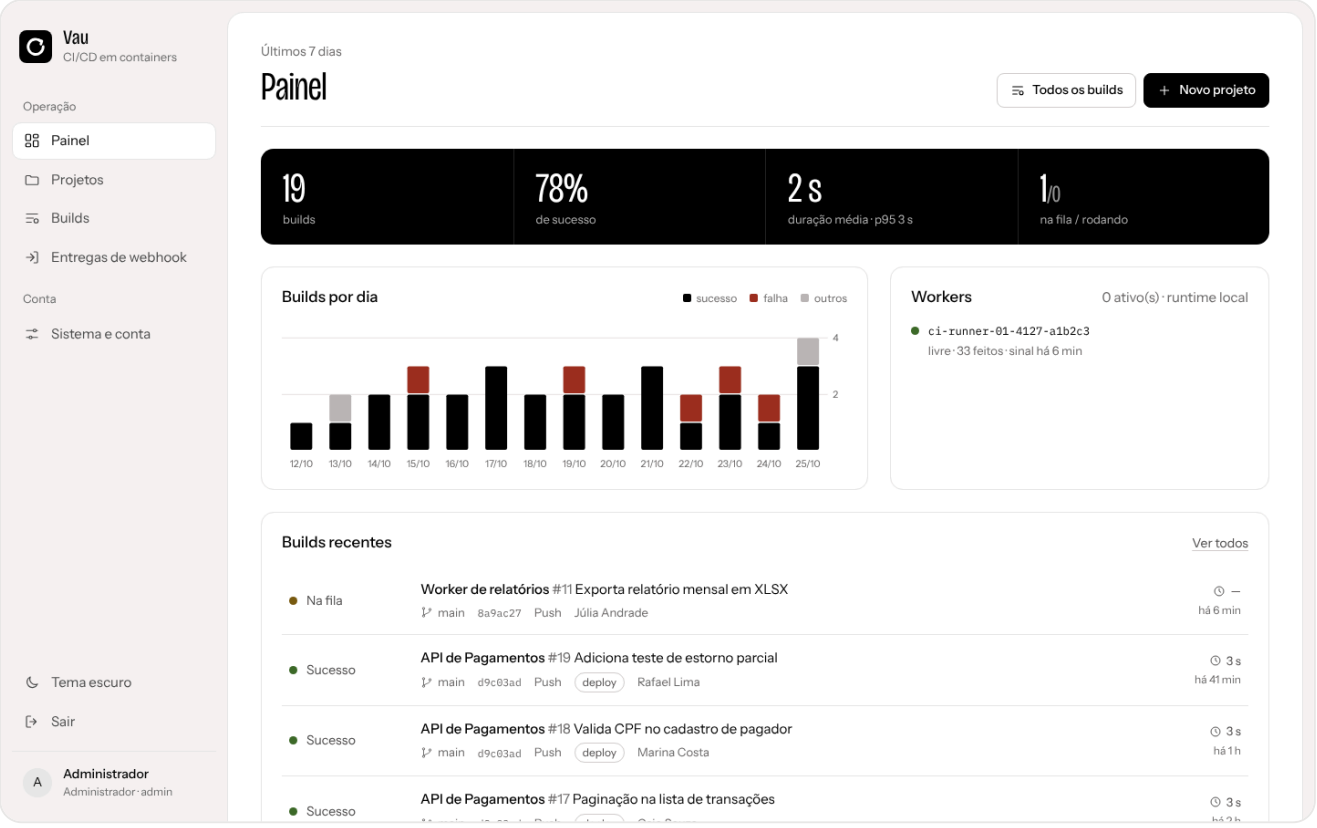


Figura 6.1 Painel: números dos últimos 7 dias na tampa preta, builds por dia, workers conectados e builds recentes.

A tampa mostra o total de builds, a taxa de sucesso (sucesso sobre sucesso + falha), a duração média com o percentil 95 e quantos builds estão na fila e rodando. O gráfico de barras empilha sucesso (preto), falha (vermelho) e outros (cinza) dos últimos 14 dias; passe o mouse para ver os números. A lista de workers mostra o estado, o build atual e o último sinal de vida.

Copyright (©) 2026 Erik Pimenta

6.2 Projetos

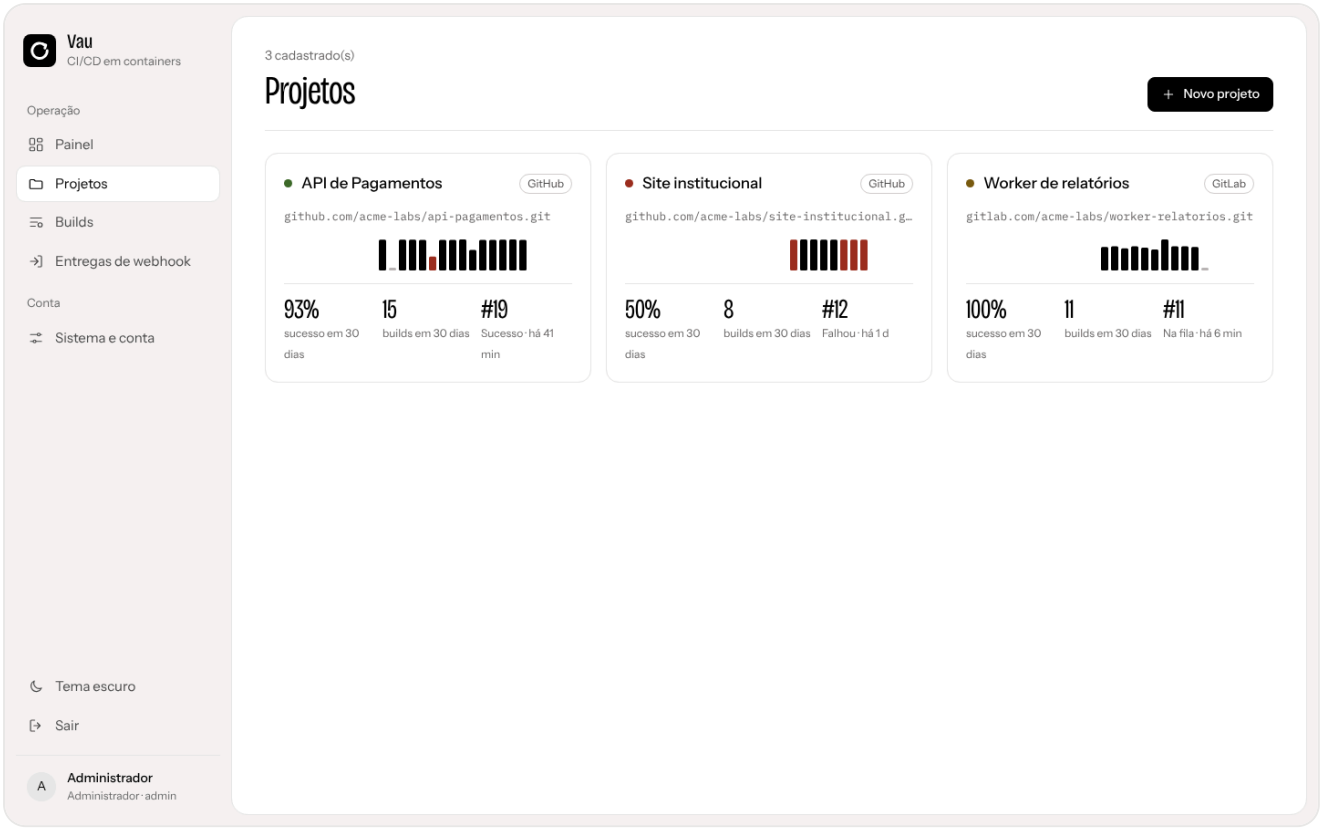


Figura 6.2 Cartões de projeto: status do último build, duração dos últimos 20 builds (cada barra, colorida pelo resultado), taxa de sucesso e total em 30 dias.

Cada projeto tem cinco abas: **Builds** (histórico), **Webhook** (URL, segredo, selo e entregas), **Pipeline** (pipeline padrão e referência rápida), **Segredos** e **Configurações**. No topo, o formulário **Executar** dispara um build manual de qualquer branch, opcionalmente num commit específico.

Copyright (©) 2026 Erik Pimenta

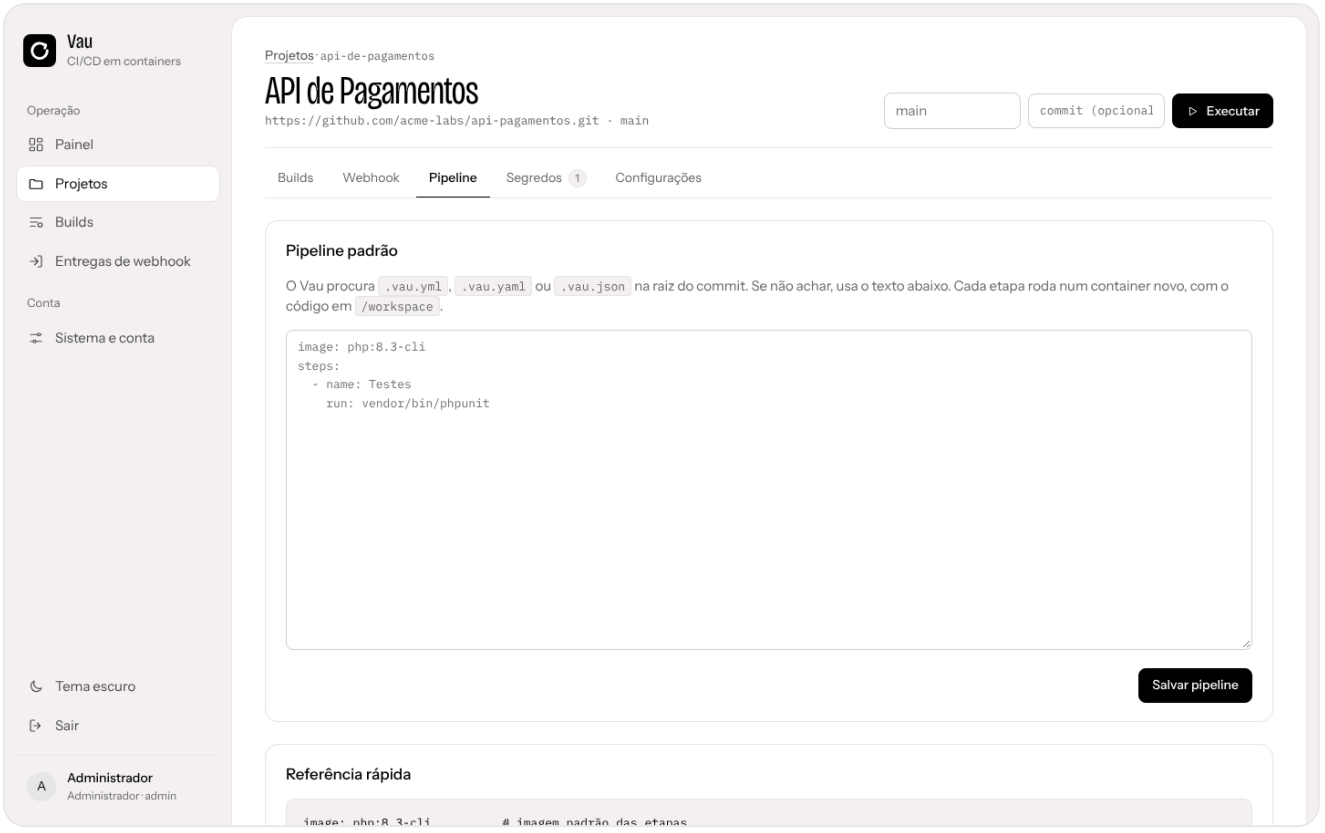


Figura 6.3 Aba Pipeline: pipeline padrão do projeto, usado quando o commit não tem .vau.yml.

Segredos

Os segredos são gravados com nome em maiúsculas e o valor nunca volta para a tela. Gravar um nome que já existe troca o valor. Marque **Somente na etapa de deploy** para credenciais de produção. Nomes começando com `VAU_` e as variáveis `CI`, `PATH` e `HOME` são reservados.

Configurações

Além dos campos do cadastro, a aba permite desativar o projeto (webhooks passam a receber 404, o histórico fica) e, para administradores, excluir o projeto digitando o identificador para confirmar.

Copyright (©) 2026 Erik Pimenta

6.3 Build e log ao vivo

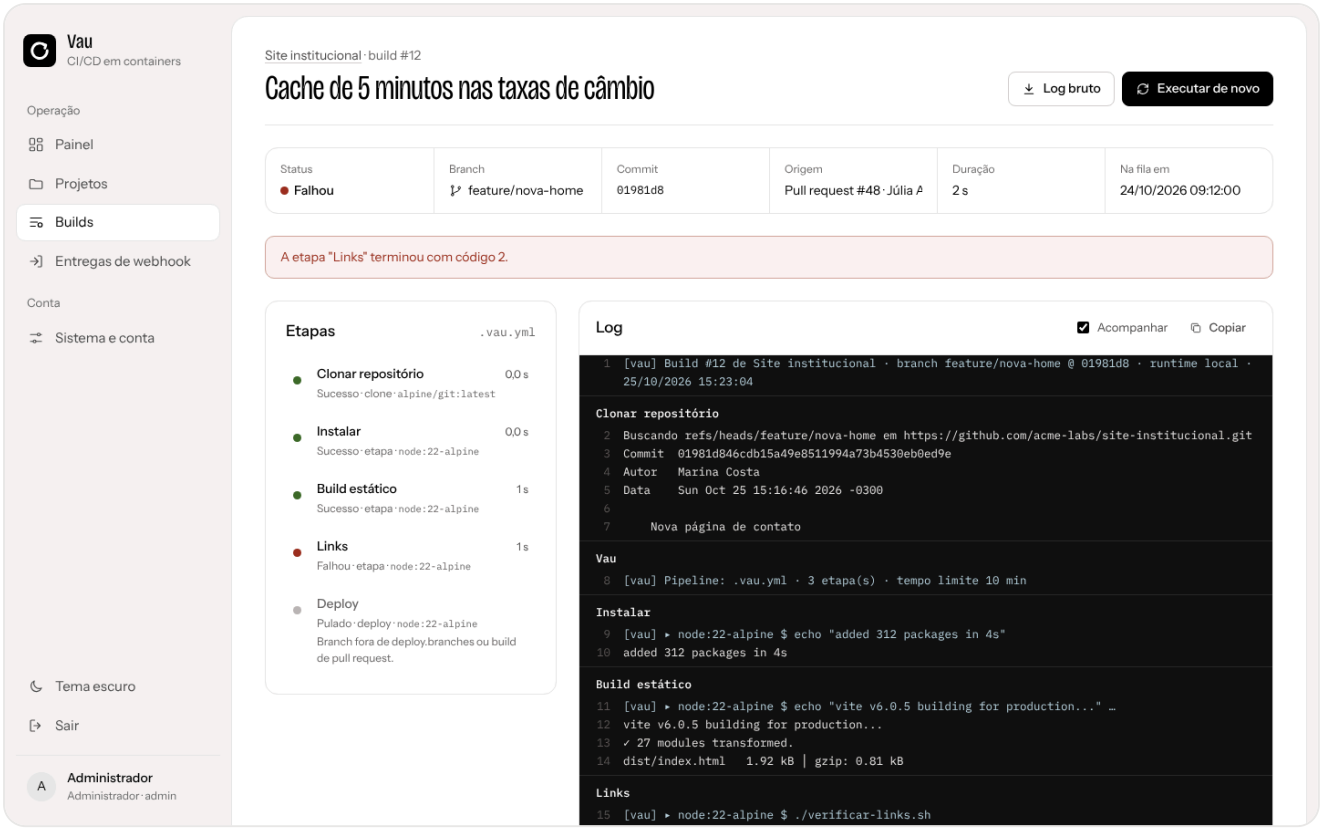


Figura 6.4 Build que falhou na etapa Links: o motivo aparece em vermelho, a etapa em falha e as seguintes como puladas, com o erro no log.

- A faixa de fatos mostra status, branch ou tag, commit (com link para o provedor), origem, duração e a hora em que entrou na fila.
- A coluna **Etapas** mostra o plano completo assim que o pipeline é lido: clone, cada etapa e o deploy, com status, imagem, duração e o motivo de pulos.
- O **Log** chega a cada segundo, separado por etapa, com linhas numeradas. Linhas de erro aparecem em vermelho claro e as mensagens do próprio Vau em azul. **Acompanhar** mantém o fim do log à vista; rolar para cima desliga.
- **Copiar** copia o log da tela; **Log bruto** abre o texto completo. **Cancelar** encerra o container em execução em até um segundo; **Executar de novo** cria um build novo com os mesmos parâmetros.

Copyright (©) 2026 Erik Pimenta

6.4 Entregas de webhook

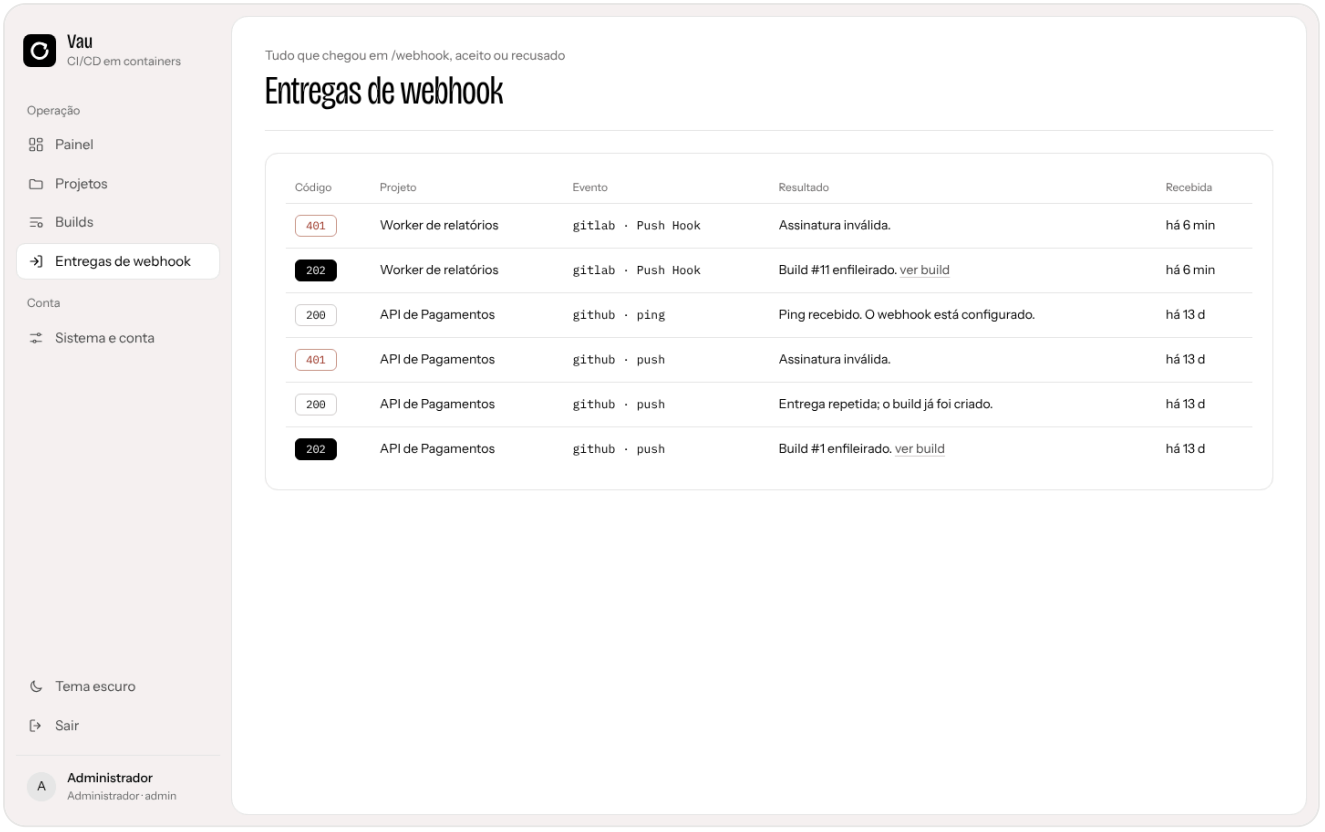


Figura 6.5 Entregas: código de resposta (202 = build criado), projeto, evento, resultado e quando chegou.

Clique numa entrega para ver cabeçalhos (com o token do GitLab mascarado) e o corpo formatado. **Reenviar** processa a entrega de novo, conferindo a assinatura outra vez; é útil depois de corrigir o filtro de branches ou reativar o projeto.

Copyright (©) 2026 Erik Pimenta

6.5 Sistema e conta

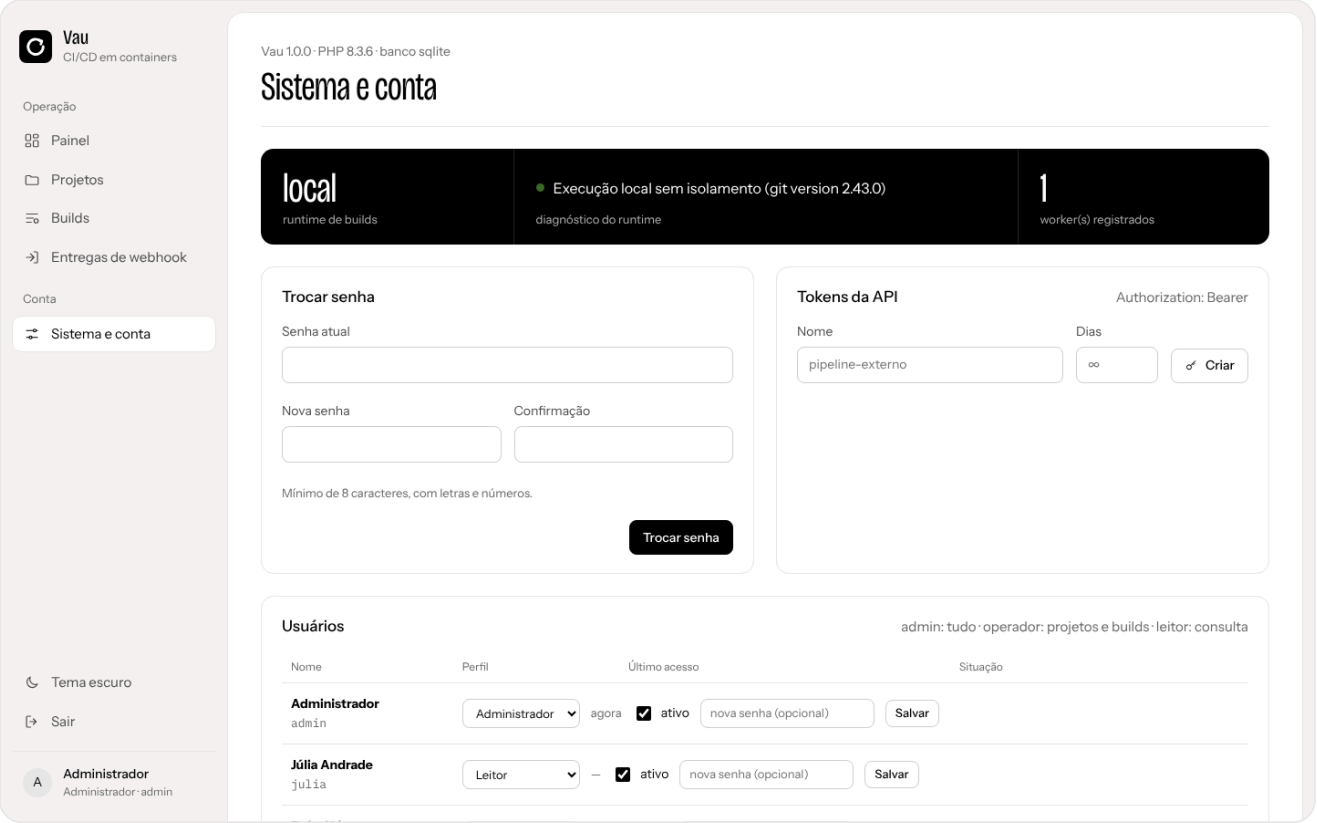


Figura 6.6 Sistema e conta: runtime e diagnóstico do Docker, troca de senha, tokens da API e, para administradores, usuários, configuração ativa e auditoria.

Perfil	Pode
Administrador	Tudo: usuários, exclusão de projetos, auditoria e configuração ativa
Operador	Criar e alterar projetos, segredos e webhooks; executar, cancelar e repetir builds; reenviar entregas
Leitor	Consultar painel, projetos, builds, logs e entregas (sem ver o segredo do webhook)

Copyright (©) 2026 Erik Pimenta

6.6 Tema escuro e celular

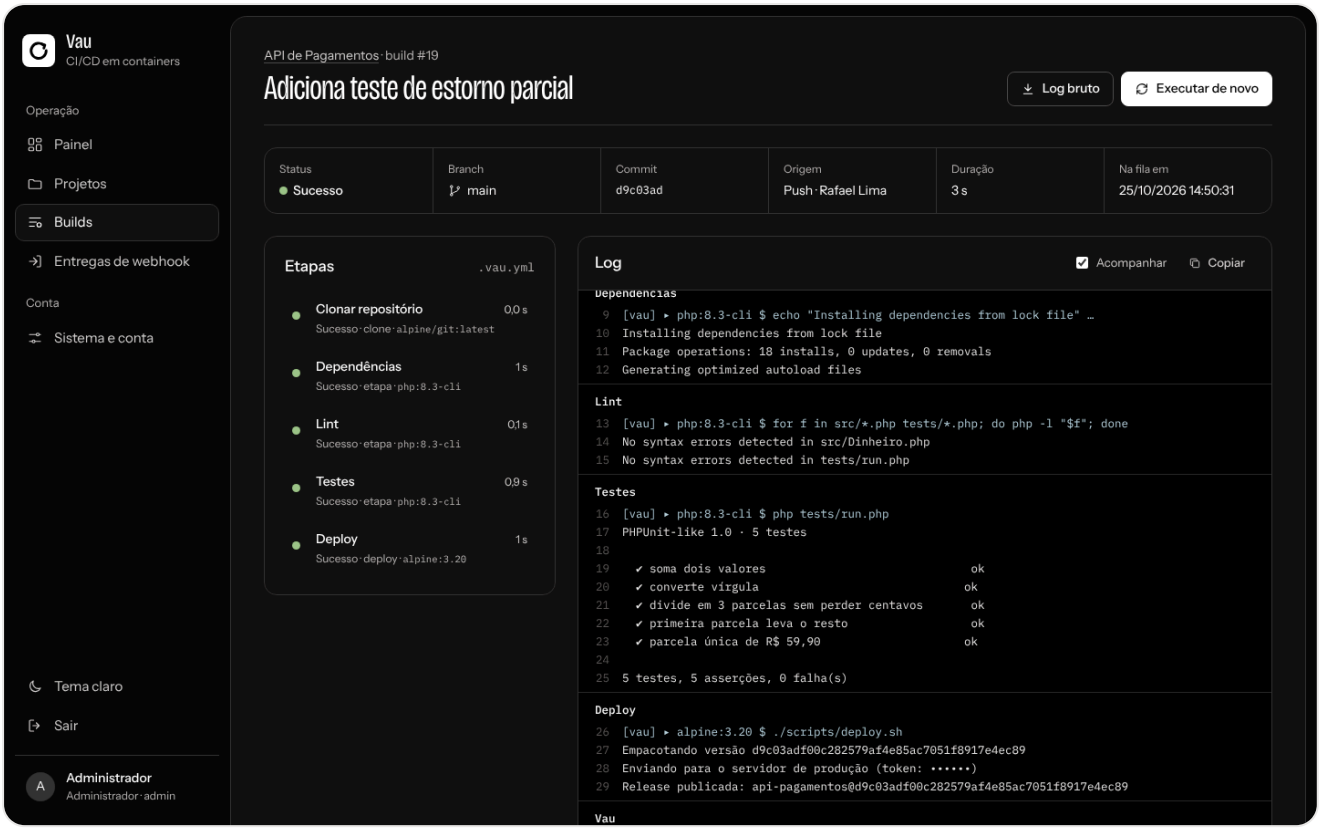


Figura 6.7 Tema escuro, escolhido no menu lateral e lembrado no navegador. Sem escolha, o painel segue o tema do sistema.

Copyright (©) 2026 Erik Pimenta



Figura 6.8 Painel no celular: o menu vira uma faixa rolável no topo.

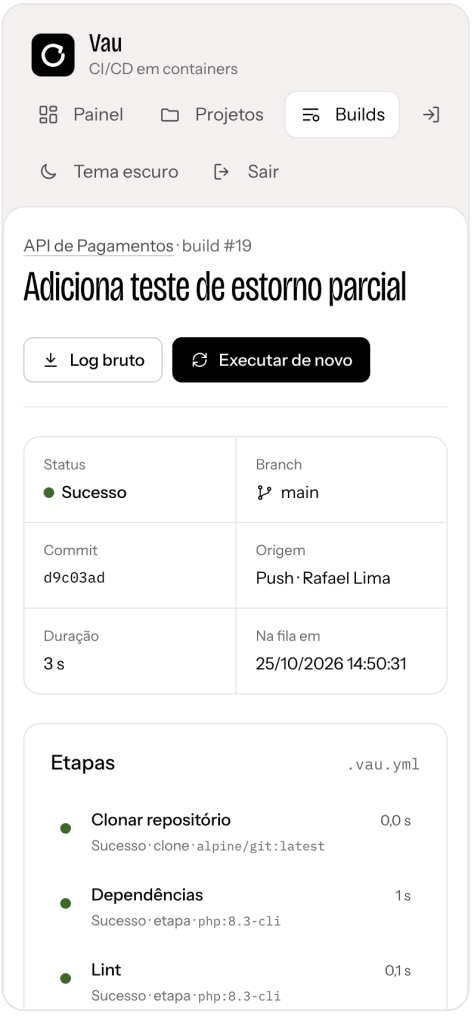


Figura 6.9 Build no celular: fatos em duas colunas e etapas acima do log.

Copyright (©) 2026 Erik Pimenta

07

Arquitetura

O Vau tem duas partes que só conversam pelo banco: o painel (que também recebe os webhooks) e os workers. Isso permite escalar cada lado separadamente e reiniciar o painel sem perder builds.

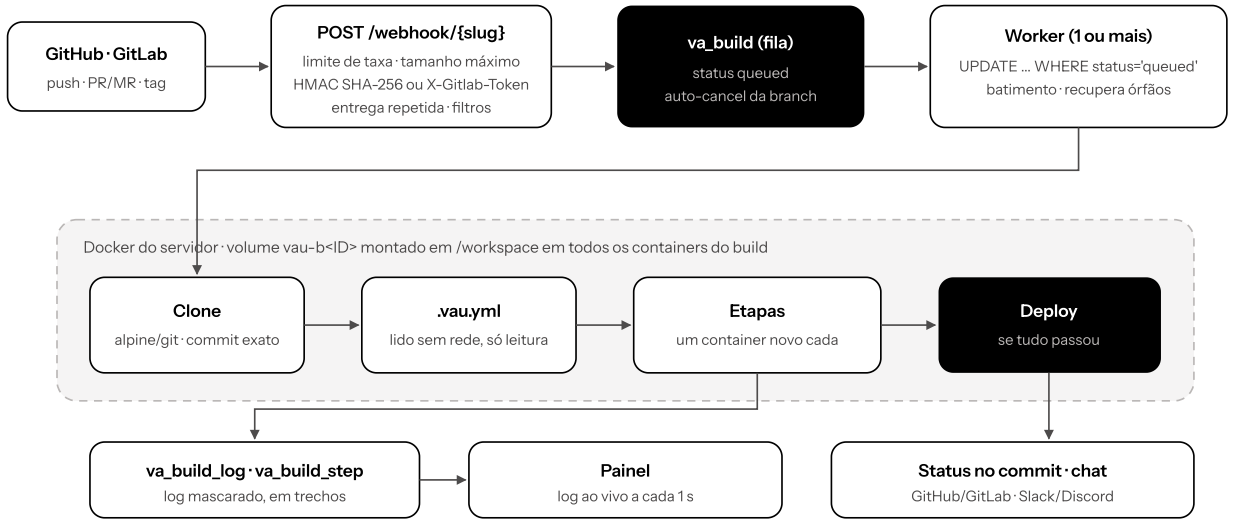


Figura 7.1 Do push ao resultado. O painel nunca executa código dos repositórios; só os workers falam com o Docker.

7.1 Módulos do código

Pasta / arquivo	Responsabilidade
`public/index.php`	Ponto de entrada HTTP do painel, da API e dos webhooks
`bin/vau`	Linha de comando (setup, worker, migrate, doctor...)
`src/App.php`, `src/Config.php`	Serviços sob demanda e leitura do `.env`
`src/Http/`	Kernel (camadas de segurança, sessão, CSRF, limite de taxa), roteador, sessão no banco, views, gráficos SVG e controladores
`src/Webhook/`	Verificação e leitura dos eventos do GitHub e do GitLab, filtros e registro das entregas
`src/Pipeline/`	Validação do `.vau.yml` e regras de deploy
`src/Build/BuildQueue.php`	Enfileirar, reservar, cancelar, repetir e recuperar builds órfãos
`src/Runner/`	Worker, execução do build, runtimes Docker e local, processos com tempo limite e gravação do log

Copyright (©) 2026 Erik Pimenta

Pasta / arquivo	Responsabilidade
`src/Integrations/`	Status no commit e avisos de chat
`src/Database/`, `src/Repository/`	Conexão PDO, migrações nos três dialetos e acesso às tabelas
`src/Support/`	Criptografia, YAML, log JSON Lines, datas e textos
`templates/`, `public/assets/`	Telas, CSS, JavaScript e fontes

7.2 Como um build é isolado

Cada build ganha um volume Docker próprio, `vau-b<ID>`, montado em `/workspace`. O clone roda num container com git; cada etapa roda num container novo, criado com `docker run --rm` e descartado no fim. Nada do build toca o disco do servidor nem os outros builds.

Proteção	Como
Container descartável	`--rm`, nome `vau-b<ID>-s<N>` e rótulo `vau.build` para limpeza garantida
CPU e memória	`--cpus`, `--memory` e `--memory-swap` iguais (sem swap extra), definidos por projeto
Processos	`--pids-limit` (`VAU_PIDS_LIMIT`) contra fork bombs
Privilégios	`--security-opt no-new-privileges` e `--cap-drop ALL` com só CHOWN, DAC_OVERRIDE, FOWNER, SETGID e SETUID de volta
Temporários	`--tmpfs /tmp` de 512 MB em memória
Rede	`bridge` ou `none` por projeto; a leitura do pipeline roda sempre sem rede e com o volume só leitura
Segredos	Entregues por `-e NOME` com o valor no ambiente do processo, nunca na linha de comando; mascarados no log, inclusive em URL e base64
Forks	Pull requests de fork não recebem nenhum segredo e nunca fazem deploy
Token do repositório	Usado só no clone e removido do `remote` logo depois
Tempo	Tempo limite do build (código 124) e cancelamento (código 130) encerram o container com `docker kill`
Limpeza	Ao fim, containers do rótulo e o volume são removidos, mesmo em erro

7.3 Fila, workers e cancelamento

- A fila é a própria tabela `va_build`. O worker reserva um build com `UPDATE ... SET status = 'running' WHERE id = ? AND status = 'queued'`; só um worker consegue mudar a linha, em qualquer banco. Rode quantos workers quiser, em quantas máquinas quiser, apontando para o mesmo banco.
- O worker registra um batimento a cada volta e, com pcntl, a cada 15 segundos durante o build. A cada 30 segundos ele encerra com erro os builds de workers sem sinal há mais de `VAU_WORKER_STALE_SECONDS`.
- SIGTERM e SIGINT terminam o build atual antes de sair. Use `--once` para processar um build e sair, ou `--max=N`.

Copyright (©) 2026 Erik Pimenta

- O cancelamento marca `cancel_requested`; o worker confere a cada segundo e encerra o container. Builds ainda na fila são cancelados na hora.
- O log é gravado em trechos de até 16 KB (ou a cada meio segundo), sem códigos ANSI, e o painel lê só o que chegou depois do último trecho.

7.4 Executando um build, passo a passo

1. O worker reserva o build e publica o status "pendente" no commit.
2. Cria o volume e clona: `git fetch --depth 50` da referência e `git checkout` do commit exato.
3. Lê `.vau.yml` (ou o pipeline padrão), valida e grava o plano de etapas, já marcando as que não rodam nesta branch.
4. Roda cada etapa com as variáveis do build, do pipeline, da etapa e os segredos. Na primeira falha, as demais são puladas.
5. Se tudo passou e a branch está liberada, roda o deploy com os segredos de deploy.
6. Remove containers e volume, grava resultado, duração e tamanho do log, publica o status final e manda o aviso de chat.

Copyright (©) 2026 Erik Pimenta

08

Segurança

As proteções estão ligadas por padrão e não dependem de configuração extra.

8.1 Proteções

Área	Proteção
Webhooks	HMAC SHA-256 (GitHub) e token (GitLab) comparados em tempo constante; corpo limitado por `VAU_MAX_PAYLOAD_KB`; limite por IP; entregas repetidas ignoradas; tudo registrado
Senhas	Argon2id (bcrypt onde não houver), mínimo de 8 caracteres com letras e números, rehash automático no login
Entrada	Bloqueio da conta após `VAU_LOGIN_MAX_ATTEMPTS` erros por `VAU_LOGIN_LOCK_MINUTES`; limite de 10 tentativas por minuto por IP; mensagem genérica que não revela se o usuário existe
Sessão	Identificador de 256 bits no cookie `vau_sessao` (HttpOnly, SameSite=Lax, Secure em HTTPS), dados no banco, troca de identificador no login, expiração por inatividade
Formulários	Token CSRF em todo POST do painel (resposta 419 sem ele)
Cabeçalhos	`Content-Security-Policy` só com a própria origem, `X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff`, `Referrer-Policy`, `Permissions-Policy` e HSTS em HTTPS
Segredos	XSalsa20-Poly1305 (libsodium) com nonce aleatório; nunca exibidos depois de gravados; fora do log técnico e da auditoria
API	Tokens Bearer guardados só como SHA-256, com validade opcional e revogação
Permissões	Perfis administrador, operador e leitor checados em cada rota
Rastro	Auditoria de entradas, alterações de projeto, segredos, builds manuais, cancelamentos, usuários e tokens, com IP

8.2 Boas práticas de implantação

- Publique o painel só em HTTPS, atrás de um proxy reverso (Nginx, Caddy, Traefik). O Vau reconhece `X-Forwarded-Proto: https`.
- Rode os workers num servidor separado da produção e, se possível, com Docker rootless ou em máquina virtual dedicada.
- Fixe a versão da imagem de clone (`VAU_CLONE_IMAGE`) e das imagens dos pipelines.
- Use rede `none` nos projetos cujos testes não precisam de internet.
- Proteja `/metrics` com `VAU_METRICS_TOKEN` se o painel for público.

Copyright (©) 2026 Erik Pimenta

09

API REST

A API usa JSON e tokens Bearer criados em Sistema e conta (ou com `php bin/vau token:create`). O token herda o perfil do usuário dono.

9.1 Rotas

Método	Rota	Função	Perfil
GET	`/api/v1/projects`	Projetos, URL do webhook, taxa de sucesso e último build	leitor
GET	`/api/v1/projects/{slug}`	Um projeto	leitor
GET	`/api/v1/projects/{slug}/builds`	Builds (`?status=` , `?branch=` , `?limit=` até 100, `?offset=`)	leitor
POST	`/api/v1/projects/{slug}/builds`	Dispara build: `{"branch": "main", "commit": "SHA"}`	operador
GET	`/api/v1/builds/{id}`	Build com as etapas	leitor
GET	`/api/v1/builds/{id}/log`	Log completo em texto	leitor
POST	`/api/v1/builds/{id}/cancel`	Cancela (202) ou informa que já terminou (409)	operador
POST	`/api/v1/builds/{id}/retry`	Repete com os mesmos parâmetros (201)	operador
GET	`/api/v1/stats`	Números do período (`?days=` de 1 a 90)	leitor
POST	`/webhook/{slug}`	Entrada dos webhooks (assinatura, sem token)	público
GET	`/health` · `/metrics` · `/badge/{slug}.svg`	Saúde, métricas Prometheus e selo	público

Copyright (©) 2026 Erik Pimenta

9.2 Exemplos com curl

```
TOKEN=vau_...   # Sistema e conta → Tokens da API

# dispara um build da branch develop
curl -s -X POST https://ci.suaempresa.com/api/v1/projects/api-de-pagamentos/builds \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"branch": "develop"}'

# acompanha o status
curl -s https://ci.suaempresa.com/api/v1/builds/42 -H "Authorization: Bearer $TOKEN"

# baixa o log
curl -s https://ci.suaempresa.com/api/v1/builds/42/log -H "Authorization: Bearer $TOKEN" > build-
42.log
```

Resposta de um build (campos em inglês, datas em UTC no formato ISO 8601):

```
{
  "data": {
    "id": 42, "project": "api-de-pagamentos", "number": 19, "status": "success",
    "trigger": "api", "branch": "develop", "tag": null, "commit": "d9c03ad...",
    "message": "Build disparado pela API", "author": null, "pull_request": null,
    "deploy": false, "queued_at": "2026-10-25T18:20:01Z", "started_at": "2026-10-25T18:20:02Z",
    "finished_at": "2026-10-25T18:20:05Z", "duration_ms": 3120, "error": null,
    "url": "https://ci.suaempresa.com/builds/42"
  }
}
```

9.3 Códigos de resposta

Código	Quando
200	Consulta bem-sucedida; webhook ignorado de propósito (ping, PR fechado, filtro, entrega repetida)
201	Build criado pela API
202	Webhook aceito e build enfileirado; cancelamento pedido
400	Webhook sem cabeçalho de provedor ou com corpo que não é JSON
401	Token da API ausente, inválido ou expirado; assinatura do webhook inválida
403	Perfil sem permissão
404	Projeto ou build inexistente; projeto desativado (webhook)
409	Cancelar um build que já terminou
413	Corpo do webhook maior que `VAU_MAX_PAYLOAD_KB`
419	Formulário do painel sem token CSRF válido
422	Dados inválidos (branch, commit, evento sem commit válido)

Copyright (©) 2026 Erik Pimenta

Código	Quando
429	Limite de requisições atingido (cabeçalho `Retry-After`)

Copyright (©) 2026 Erik Pimenta

10

Operação

Comandos, monitoramento, limpeza e backup.

10.1 Linha de comando

Comando	Função
`php bin/vau setup`	Prepara pastas, chave, banco e o usuário admin (rode uma vez; repetir é seguro)
`php bin/vau serve [--host= --port=]`	Painel no servidor embutido do PHP
`php bin/vau worker [--once] [--max=N]`	Consome a fila de builds
`php bin/vau migrate`	Aplica as migrações pendentes
`php bin/vau doctor`	Confere PHP, extensões, banco, permissões e o runtime (sai com código 1 se algo falhar)
`php bin/vau user:create <usuario> <perfil> [--nome=]`	Cria usuário (pede a senha)
`php bin/vau user:password <usuario>`	Redefine a senha e remove bloqueios
`php bin/vau token:create <usuario> [nome] [--dias=N]`	Cria token da API e imprime o valor
`php bin/vau build:run <projeto> [--branch=] [--commit=]`	Enfileira um build
`php bin/vau build:cancel <id>`	Cancela um build
`php bin/vau prune [--dias=N]`	Apaga builds terminados e entregas antigos
`php bin/vau schema:dump <sqlite|mysql|pgsql>`	Imprime o DDL do banco
`php bin/vau key:generate`	Gera um valor para `VAU_APP_KEY`

Copyright (©) 2026 Erik Pimenta

10.2 Saúde e métricas

```
$ curl -s http://127.0.0.1:8080/health
{ "status": "ok", "versao": "1.0.0", "checks": { "banco": "ok", "workers_ativos": 2, "fila": 0 }
}

$ curl -s http://127.0.0.1:8080/metrics
vau_builds{project="api-de-pagamentos",status="success"} 12
vau_queue_depth 0
vau_workers_alive 2
vau_build_duration_seconds_avg 3.412
vau_webhook_deliveries_total{code="202"} 31
```

`/health` responde 503 se o banco não responder, o que serve para o healthcheck do Docker e do balanceador. `/metrics` segue o formato de exposição do Prometheus e também traz `vau_info` com a versão.

10.3 Logs

O log técnico fica em `storage/logs/vau.log`, uma linha JSON por evento (data em UTC, nível, mensagem, contexto e PID), pronto para Loki, Elastic ou CloudWatch. Cada resposta HTTP leva o cabeçalho `X-Request-Id`; erros 500 mostram esse código na tela para facilitar a busca no log. O log dos builds fica no banco.

10.4 Limpeza e backup

```
# todo dia às 03:10: apaga builds e entregas com mais de VAU_LOG_RETENTION_DAYS dias
10 3 * * * cd /opt/vau && php bin/vau prune >> storage/logs/prune.log 2>&1

# backup do SQLite com o Vau rodando
sqlite3 storage/vau.sqlite ".backup storage/backup-$(date +%F).sqlite"
```

Guarde junto o arquivo `storage/.chave` (ou o valor de `VAU_APP_KEY`), em local separado. O manual **Banco-de-Dados-Vau.pdf** traz o backup de MySQL e PostgreSQL e a migração entre bancos.

Copyright (©) 2026 Erik Pimenta

11

Qualidade e manutenção

Testes automatizados, análise estática, estilo de código, integração contínua e o registro diário de atividade.

11.1 Testes automatizados

```
composer install
composer test        # PHPUnit
composer analyse     # PHPStan
composer lint        # PHP-CS-Fixer (só confere)
composer check       # os três
```

São 49 casos de teste em PHPUnit 11, com SQLite em memória, runtime de containers falso e cliente HTTP falso, sem Docker nem rede. Cobrem o leitor de YAML, a validação do pipeline e as regras de deploy, a assinatura e a leitura dos eventos do GitHub e do GitLab, a fila (reserva exclusiva, cancelamento automático, repetição), o build completo (etapas, deploy, segredos só de deploy, mascaramento no log, forks sem segredos, falha de clone, pipeline ausente, cancelamento), o painel (entrada, bloqueio, CSRF, criação de projeto e todas as telas), a API com token, as rotas públicas e os cabeçalhos de segurança.

Ferramenta	Configuração	Regra
PHPUnit 11	`phpunit.xml.dist`	Suítes unit e integration; falha em avisos e testes arriscados
PHPStan 2	`phpstan.neon.dist`	Nível 6 em `src/` e `bin/vau`
PHP-CS-Fixer	`.php-cs-fixer.dist.php`	PER-CS 2.0, `declare(strict_types=1)`, imports ordenados
EditorConfig	`.editorconfig`	UTF-8, LF, 4 espaços (2 em YAML, JSON, CSS e JS)

11.2 Integração contínua

O workflow `.github/workflows/ci.yml` roda em todo push na `main`, em pull requests, todo dia às 10:07 (Brasília) e sob demanda:

- **PHP 8.2, 8.3 e 8.4**: instalação com cache do Composer, checagem de sintaxe, estilo (no 8.3), PHPStan, PHPUnit e um teste de fumaça da linha de comando.
- **PostgreSQL 16**: setup e migrações num banco real.
- **Imagem Docker**: build da imagem e conferência de que `/health` responde.

O Dependabot (`.github/dependabot.yml`) abre pull requests semanais para Composer e GitHub Actions e mensais para a imagem Docker.

Copyright (©) 2026 Erik Pimenta

11.3 Commit automático diário

O workflow `.github/workflows/diario.yml` registra cinco commits por dia no arquivo `.github/atividade.txt`, em horários espalhados (Brasília):

Horário	Turno	Cron (UTC)
08:13	manhã	`13 11 * * *`
10:47	fim da manhã	`47 13 * * *`
13:29	início da tarde	`29 16 * * *`
16:11	tarde	`11 19 * * *`
19:37	noite	`37 22 * * *`
21:58 e 23:21	reservas	`58 0 * * *` e `21 2 * * *`

O GitHub pode atrasar ou pular execuções agendadas em horários de pico. Por isso cada execução grava o primeiro horário do dia que já passou e ainda não foi registrado, e as duas reservas recuperam o que se perdeu: os cinco commits saem mesmo que duas execuções falhem, e nunca saem repetidos. O CI ignora mudanças nesse arquivo.

1. No repositório, abra **Settings** → **Secrets and variables** → **Actions**.
2. Crie `GIT_NAME` com o seu nome e `GIT_EMAIL` com o e-mail da sua conta do GitHub (o endereço `...@users.noreply.github.com` serve), para os commits contarem no seu perfil.
3. Em **Settings** → **Actions** → **General** → **Workflow permissions**, escolha **Read and write permissions**.
4. Em **Actions** → **Atividade diária** → **Run workflow**, rode uma vez para conferir.

11.4 Solução de problemas

Sintoma	Causa provável	Solução
Build parado em "Na fila"	Nenhum worker rodando	Rode `php bin/vau worker` e confira o painel (Workers) ou `/health`
"Docker não respondeu" no diagnóstico	Daemon parado ou usuário fora do grupo docker	`docker info` com o mesmo usuário do worker; no Windows, abra o Docker Desktop
Webhook com 401	Segredo diferente no provedor	Copie de novo o segredo na aba Webhook; no GitHub, use `application/json`
Webhook com 404	Slug errado ou projeto desativado	Confira a URL e a situação do projeto
Webhook 200 sem build	Evento desligado, filtro de branches ou entrega repetida	Leia o resultado em Entregas; ajuste e use Reenviar
Falha ao clonar	Repositório privado sem token, ou token sem permissão	Informe o token com leitura do repositório
"Nenhum pipeline encontrado"	Commit sem `.vau.yml` e projeto sem pipeline padrão	Crie o arquivo na raiz ou cadastre na aba Pipeline

Copyright (©) 2026 Erik Pimenta

Sintoma	Causa provável	Solução
Etapa termina com código 137	Memória do container esgotada	Aumente a memória do projeto
Etapa termina com código 124	Tempo limite atingido	Aumente o tempo do projeto ou o `timeout` do pipeline
Status não aparece no commit	Sem token ou opção desligada	Token com permissão de commit statuses e opção marcada
Builds "O worker parou de responder"	Worker reiniciado no meio do build	Normal após quedas; use Executar de novo
Conta bloqueada	Senhas erradas seguidas	Aguarde, ou desbloqueie em Sistema e conta, ou `php bin/vau user:password`
Erro ao decifrar segredo	A chave mudou	Restaure `storage/.chave` ou a `VAU_APP_KEY` original

Copyright (©) 2026 Erik Pimenta